Penetration Testing 1 SQL INJECTION: Web Application SQL Injection and Authentication Bypass Project: StaffHub Employee Portal (SQL Injection Analysis)

Target: localhost (p-8080)

Tester: Surakshya Chhetri

Date of Assessment: August, 2026

1. Overview

The documentation includes all the vulnerabilities and steps for SQL injection vulnerabilities. When I examined SQL injection with login functionality, I checked whether user input was secured or not. I inspected the authentication mechanism (with username and password).

While checking database security, I logged in using SQL syntax in its query, creating a potential authentication bypass. The report documents all the results against a web application for SQL injection conducted within the secure environment.

Vulnerability

- **SQL Injection – Login Bypass**
- **Vulnerability Rate – Critical**
- Authentication bypass and potential database compromise.
- OS Command Injection

Testing Performed — Scope

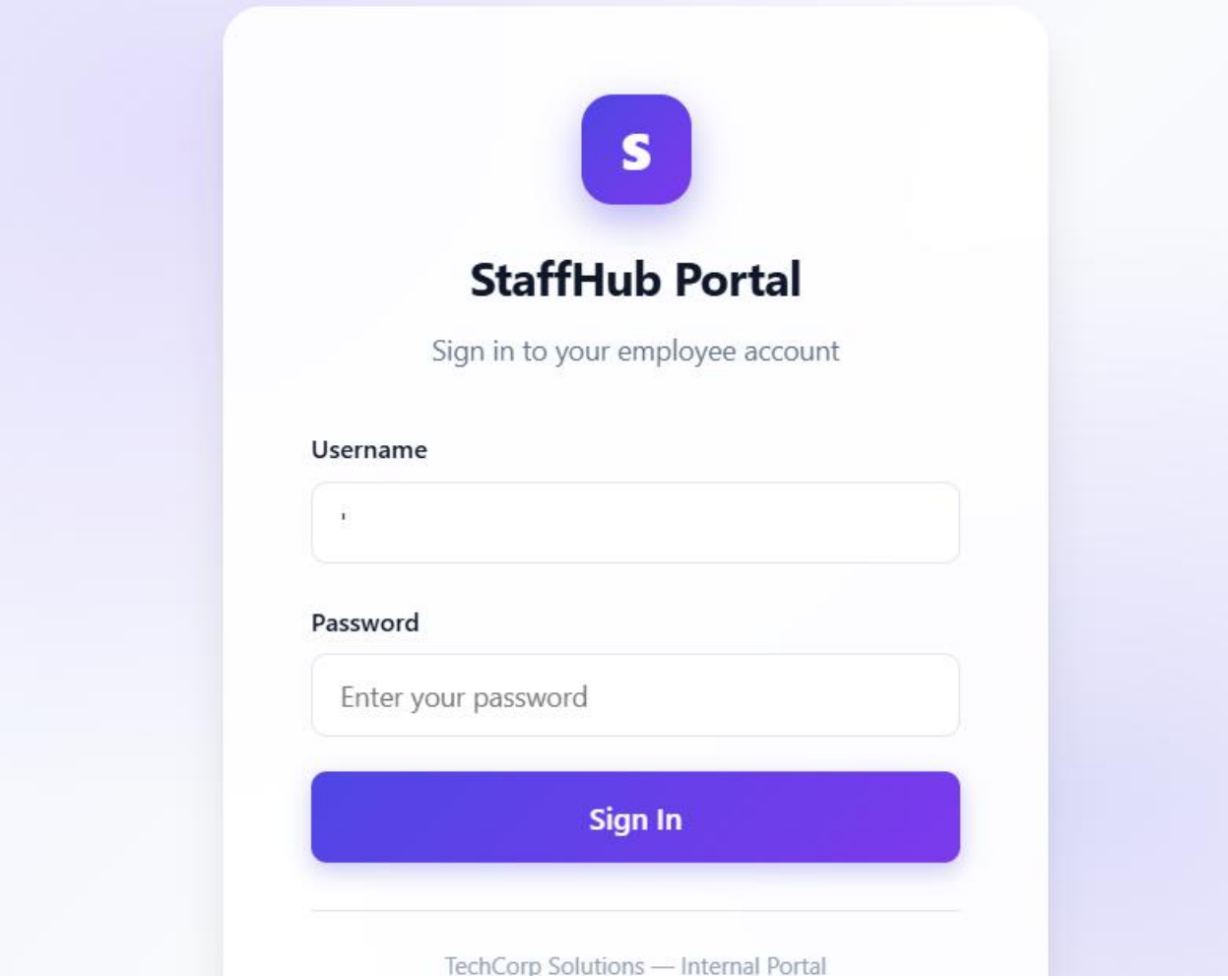
1. Login and authentication functionality
2. Username and password input
3. SQL query handling
4. Bypass behavior
5. SQL injection testing
6. Administrative portal access
7. Information exposure

Objective

1. To see whether application inputs were vulnerable.
2. To check whether SQL injection affects authentication.
3. To bypass authentication controls and manipulate database queries.
4. To check whether database information could be exposed.
5. To see overall security.

Methodology

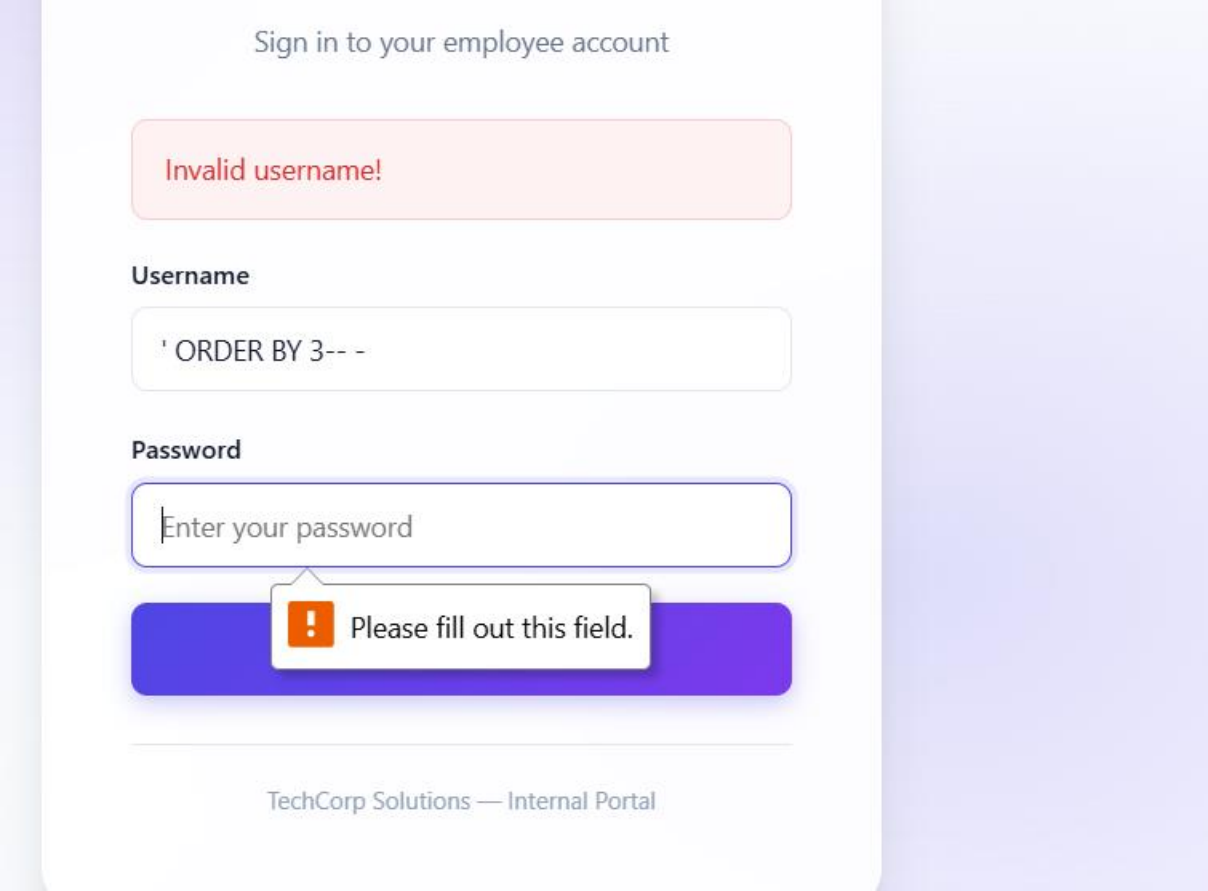
Firstly, I started penetration testing by following a secured environment. I continued to check the application's authentication entry points and observed the behavior of the login functionality. I tested using random usernames and passwords to see normal functionality of login, and after that, using SQL test cases, I compared responses of the application with login behavior.



I used the "ORDER BY 1" to identify abnormal behavior. Adding UNION SELECT as an SQL query allowed for major injection testing.

Initial Testing and Authentication Bypass

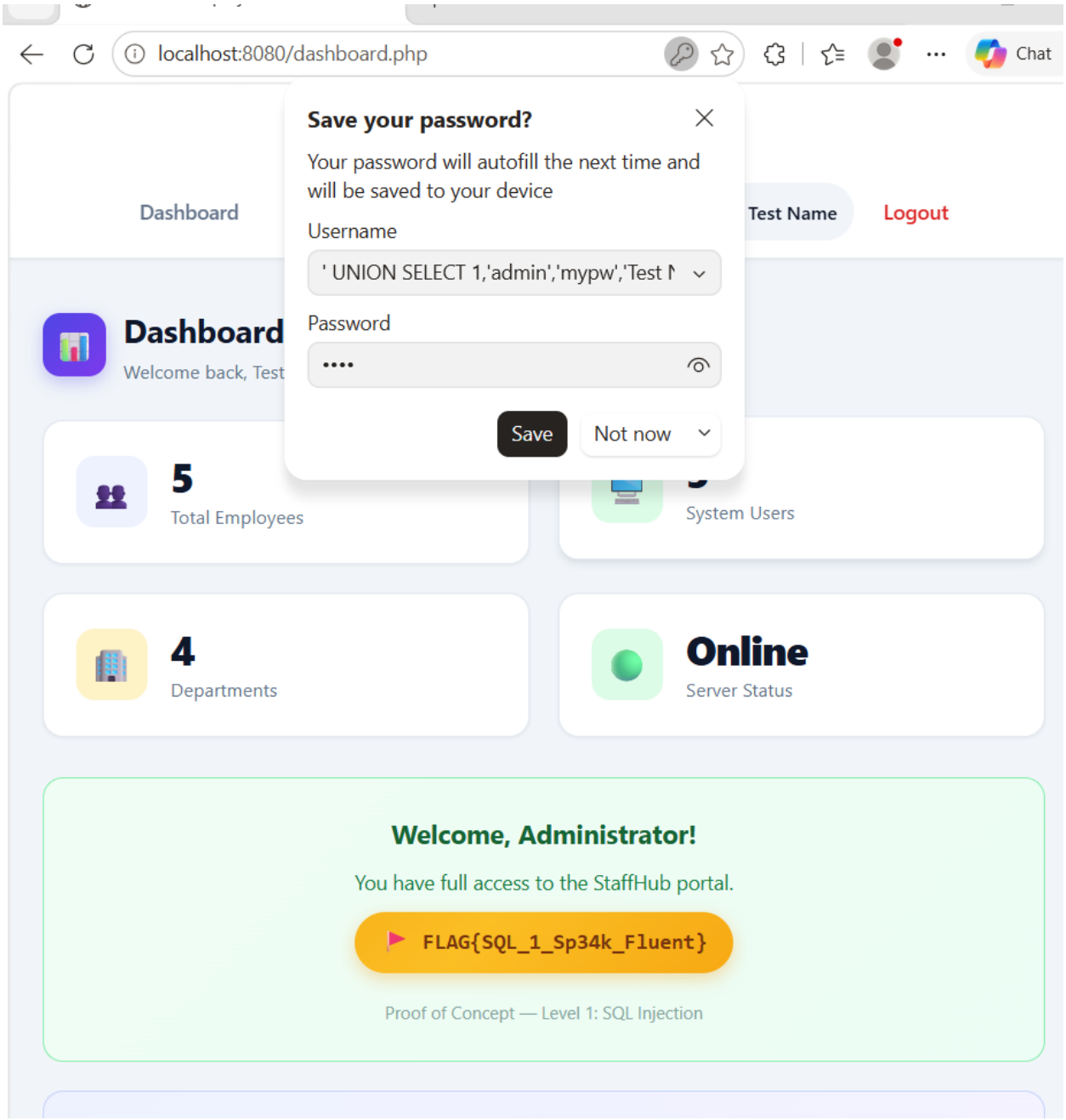
Process: Testing the username and password fields, trying to influence login with SQL syntax. Using "ORDER BY" for identifying the number of columns.



Output: Finding SQL vulnerability and authentication bypass.

UNION SELECT

I performed union-based SQL injection proof of concept to inspect whether the application's query could return database information. The test was for validation. The union injection is based on the number of columns. For example, if the column number is 6, database information can be retrieved.



After the SQL injection, I checked if information could be returned through requests.

Portal Access

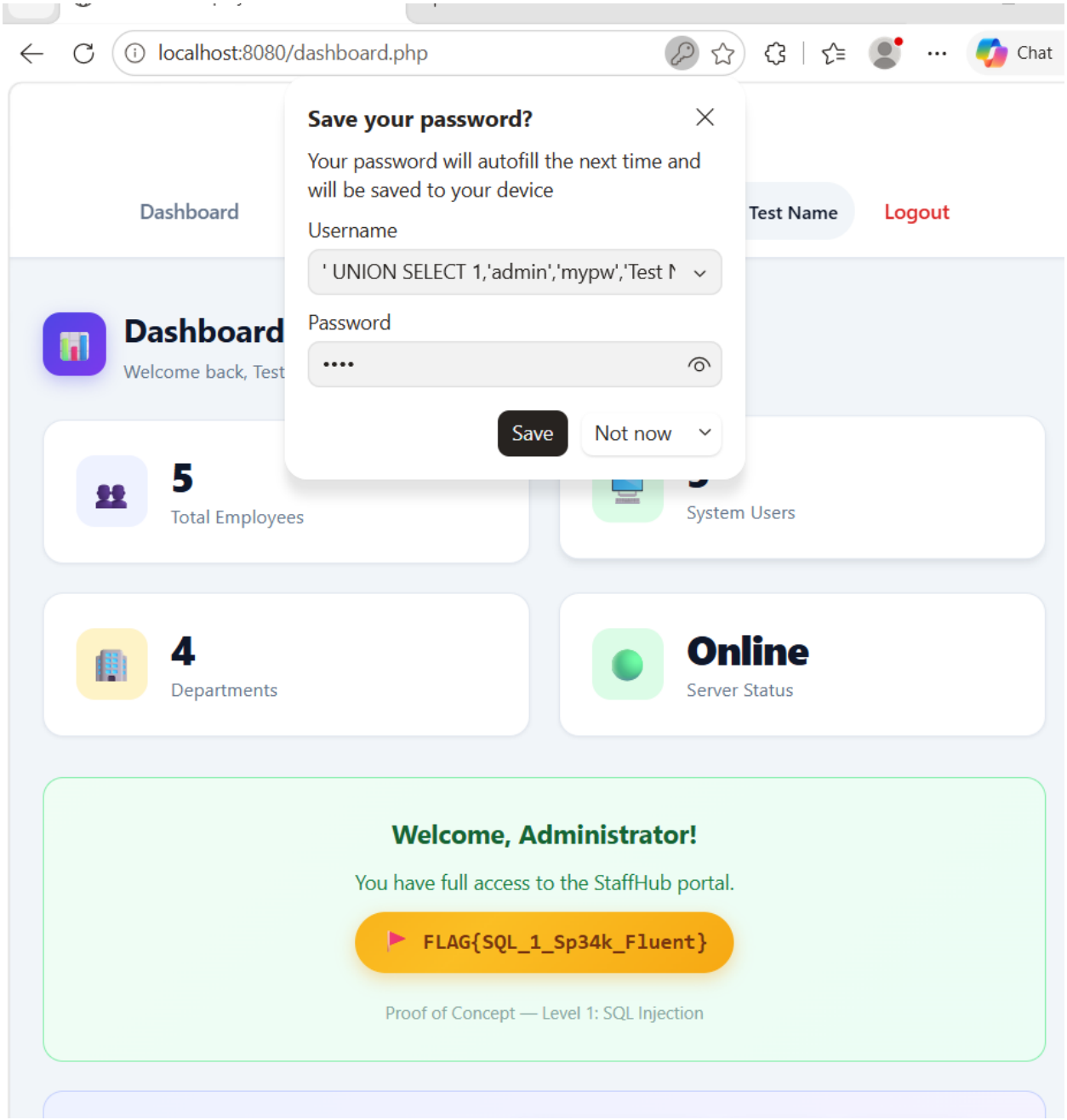
After all the steps, it shows an unauthorized party can reach privileged application.

Result

Flag:sql injection

status: positive

severity : critical



Summary and risk

The assessment confirmed a serious SQL injection vulberability in the given application. The vulnerability is a high risk as it can affect multiple security at once including all the database and administrative access from where the attacker can easily manipulate or change the database queries from which crucial information can be easily accessed.

Recommanded remediation

- Use parameterized queries.
- Apply server side input validation
- Do not share database errors to users
- Minimize privilege database accounts

Conclusion

This test comfrim that SQL injection could affect the applications authentication as well as database functionality through bypass and can get privileged access.